Scoops and Smiles - User Manual | Ice Cream Shop Management System

Sofia DePasquale, Chloe Kress, Jenell Hall, Josh VanNostrand, Jack TeBeau, Andrew Rautzhan

---

Table of Contents

- Getting Started - First time setup
- Opening Database Tool Window - Detailed DB Browser setup
- Using the System - How to use each tab
- Generating Reports - Income statements and analysis
- Troubleshooting - Common issues
- Quick Reference - Essential information

Getting Started

Your zip file contains:

- **scoop_system_v2.py** - Main application (67 KB)
- **generate_data.py** - Sample data generator (5 KB)

System Requirements

- **Python 3.x** (installed with PyCharm)
- **PyCharm IDE** (any version)
- **No additional packages needed!** (Uses standard libraries only)

First Time Setup

Step 1: Extract the Files

1. Unzip ScoopSystem.zip to a folder:

```
None
C:\Users\sofid\OneDrive\Documents\MGIS350\ScoopSystem\
```

2. Your folder should contain:

```
None
📁 ScoopSystem
  ├── scoop_system_v2.py
  └── generate_data.py
```

Step 2: Open Project in PyCharm

1. Open **PyCharm**
2. Click **File → Open**
3. Navigate to your ScoopSystem folder
4. Click **OK**

Step 3: Run the Application

1. In the **Project** window (left side), double-click **scoop_system_v2.py**

2. Click the green **Run** button (▶) at the top right

3. You should see in the console:

```
None
Connected to database: scoops_smiles.db
Tables created successfully
```

Step 4: A window opens showing 6 tabs:

- Sales Entry
- Ice Cream Purchase
- Supply Purchase
- Current Inventory
- Income Statement
- Flavor Analysis

Success! The database file scoops_smiles.db is created in your project folder

Step 4: Add Sample Data (Recommended)

1. Close the application window (X button)
2. In PyCharm, double-click **generate_data.py**
3. Click **Run**
4. Type yes when prompted
5. Wait 5-10 seconds
6. You should see:

```
None
✓ Generated 3,000+ sales records
✓ Generated 150+ inventory purchase records
Sample data generation complete!
```

Now you have realistic test data.

Opening Database Tool Window

Why Use the Database Browser?

The Database Browser in PyCharm lets you:

- View all your data in organized tables
- Run SQL queries to analyze data
- Verify sales and purchases were recorded correctly
- Explore the database structure

Step-by-Step Setup

Step 1: Open Database Window

Choose one method:

Method 1 - Menu:

- Click **View → Tool Windows → Database**

Method 2 - Sidebar:

- Look at the **left edge** of PyCharm

- Click the vertical **Database** tab

Method 3 - Keyboard:

- Press Alt + 1, then click **Database** tab on the right

Step 2: Add Database Connection

1. In Database window, click the + (plus) button
2. Hover over **Data Source** → Click **SQLite**
3. A dialog box opens: "Data Sources and Drivers"

Step 3: Configure Connection

In the configuration dialog:

1. **Name:** Type Scoops and Smiles

2. **File:**
    - Click the folder icon 📁
    - Navigate to: C:\Users\sofid\OneDrive\Documents\MGIS350\ScoopSystem\
    - Select: scoops_smiles.db
    - Click **Open**

3. **Driver:**
    - Should say "SQLite"
    - If it says "Download driver files", click it and wait 10-20 seconds

4. **Test Connection:**
    - Click **Test Connection** at bottom
    - Should say "Connection successful"
    - If error: Make sure Python app is closed and database file exists

5. Click **Apply**, then **OK**

Step 4: Connect and View Data

1. In Database window, you'll see: Scoops and Smiles

2. Click the **plug icon** to connect (turns green)

3. Click arrow next to "Scoops and Smiles" → **main** → **Tables**

4. You'll see 6 tables:
    - locations
    - prices
    - current_inventory
    - sales
    - inventory_purchases
    - supply_purchases
5. **Double-click any table** to view its data

Step 5: Running SQL Queries (Optional)

1. Right-click on **main → New → Query Console**
2. Type a query and press Run (▶)

Example queries:

```SQL
-- View all locations
SELECT * FROM locations;

-- Count sales by location
SELECT l.location_name, COUNT(*) as total_sales
FROM sales s
JOIN locations l ON s.location_id = l.location_id
GROUP BY l.location_name;

-- Check inventory levels
SELECT l.location_name, ci.flavor, ci.quantity_ounces
FROM current_inventory ci
JOIN locations l ON ci.location_id = l.location_id
ORDER BY ci.quantity_ounces ASC;
```

Step 6: Disconnect When Done

Important: Only ONE program can use the database at a time!

- To use Python app: Disconnect DB Browser (click plug 🔌)
- To use DB Browser: Close Python app first

Using the System

Tab 1: Sales Entry

Records individual ice cream sales

How to add a sale:

1. **Date:** YYYY-MM-DD format (defaults to today)
2. **Location:** Select store (Downtown Rochester, Brighton, or Henrietta)
3. **Size:** Kiddie ($3), Small ($3.50), Medium ($4), Large ($4.50)
4. **Flavor:** Vanilla, Chocolate, Cookies & Cream, Neapolitan, Cookie Dough
5. **Container:** Standard Cone, Waffle Cone, or Dish with Spoon
6. Click **Add Sale**
7. Confirm in dialog
8. **Result:** Sale saved, inventory decreases automatically

Features:

- Recent Sales list shows last 20 sales
- Delete Selected Sale button (restores inventory)
- Clear Form button

Tab 2: Ice Cream Purchase

Records bulk ice cream purchases

How to add a purchase:

1. **Date:** Today or past date
2. **Location:** Which store receives ice cream
3. **Flavor:** Select flavor (cost shows automatically)
4. **Quantity:** Number of 5-gallon containers (each = 640 oz)
5. Click **Add Purchase**
6. Confirm
7. **Result:** Purchase saved, inventory increases

Features:

- Recent Purchases list shows last 20
- Delete Selected Purchase button (adjusts inventory)

- Clear Form button

Tab 3: Supply Purchase

Records container and napkin purchases

How to add:

1. **Date:** Today or past
2. **Location:** Which store
3. **Supply Type:** Cones ($5-$6 per 100) or Napkins ($20 per 10,000)
4. **Quantity:** Number of units
5. Click **Add Supply Purchase**
6. Confirm
7. **Result:** Purchase saved

Features:

- Recent Supply Purchases list
- Delete Selected Purchase button
- Clear Form button

Tab 4: Current Inventory

Real-time ice cream stock levels

What you see:

- Location, Flavor, Ounces, Gallons, Status
- **OK (Green)** = Above 10 gallons
- **LOW STOCK (Red)** = Below 10 gallons

How inventory updates:

- **Increases:** Ice cream purchases (Tab 2)
- **Decreases:** Sales (Tab 1)
- **Restores:** Deleting sales
- **Removes:** Deleting purchases

Features:

- Refresh button
- Check Low Stock button (manual check)
- Automatic low stock alert on startup

Tab 5: Income Statement

Monthly financial reports

How to generate:

1. **Select Locations:** Check boxes for locations to include
2. **Select Month/Year:** Choose from dropdowns
3. Click **Generate Income Statement**
4. Report appears

What the report shows:

```
None
Downtown Rochester - November 2024
═══════════════════════════════════════════════

Revenue:                        $15,234.50
  Less: Cost of Goods Sold        $3,145.23
Gross Profit:                   $12,089.27

Operating Expenses:
  Rent, Utilities, Labor, Equip.  $18,250.00
Net Income:                      $(6,160.73)
═══════════════════════════════════════════════
```

Understanding:

- **Revenue** = Total sales
- **COGS** = Cost of ice cream, containers, napkins SOLD (not purchased!)
- **Gross Profit** = Revenue - COGS
- **Operating Expenses** = Fixed costs ($18,250/month per location)
- **Net Income** = Gross Profit - Operating Expenses

Critical concept - Perpetual Inventory:

- COGS only includes costs from actual sales

- Inventory purchases are NOT expenses (they're assets)
- This prevents double-counting
- Matches GAAP accounting standards

Features:

- Export Report button (save as text file)
- Combined Report option (totals across locations)

Tab 6: Flavor Analysis

Shows popular flavors

How to generate:

1. Select month and year
2. Click **Generate Flavor Report**
3. Report shows for each location and overall:
    - Flavor name
    - Scoops sold
    - Ounces sold
    - Revenue

Use this to:

- Plan inventory orders
- Stock more of bestsellers
- Make marketing decisions

Generating Reports

Income Statement

Steps:

1. Income Statement tab
2. Check locations
3. Select month/year
4. Generate

5. Export (optional)

Key Point: COGS only includes sales costs, NOT inventory purchases. This is the perpetual inventory system and prevents double-counting.

Flavor Analysis

Steps:

1. Flavor Analysis tab
2. Select month/year
3. Generate
4. Review which flavors sell best

Expected Q&A

Q: How does inventory work? A: "Tracks ounces. Sales decrease it, purchases increase it. Alerts when below 10 gallons."

Q: Why not include purchases in COGS? A: "Perpetual inventory system. Purchases are assets, not expenses. Only sales costs go in COGS. Prevents double-counting, matches GAAP."

Q: What validation exists? A: "Date format, no future dates, positive numbers only, database integrity checks."

Q: Can it crash? A: "No. All database operations use try-except blocks. Errors show messages, app stays running."

Troubleshooting

Application Issues

Won't start:

- Check Python installed (File → Settings → Project → Python Interpreter)
- Look for errors in console
- Restart PyCharm

"Database is locked":

- Close either Python app OR DB Browser
- Only one can use database at a time

Database Browser Issues

"Session disconnected":

- Close Python app
- Disconnect DB Browser (click plug)
- Reconnect

Can't find database:

- Run scoop_system_v2.py once to create it
- Check file exists in project folder

Data Entry Issues

Date field red:

- Use YYYY-MM-DD format
- No future dates
- Valid dates only (e.g., 2024-02-30 is invalid)

Quantity field red:

- Must be positive number
- No negatives, no zero
- No text

"Insufficient inventory":

- Not enough ice cream
- Add purchase first (Tab 2)
- Or click Yes to continue anyway

Report Issues

No data:

- Check correct month/year selected
- Make sure sales exist for that period
- Run generate_data.py for sample data

Numbers look wrong:

- COGS should be 15-25% of revenue

- Net Income often negative (high fixed costs)
- Need ~$21,800/month revenue to break even

Quick Reference

Pricing

Sale Prices:

- Kiddie (4 oz): $3.00
- Small (8 oz): $3.50
- Medium (12 oz): $4.00
- Large (16 oz): $4.50

Ice Cream Costs (per 5-gal = 640 oz):

- Vanilla: $1.80
- Chocolate: $1.80
- Cookies & Cream: $1.90
- Neapolitan: $1.85
- Cookie Dough: $1.95

Operating Expenses (per location/month):

- Rent: $1,000
- Utilities: $250
- Labor: $15,000
- Equipment: $2,000
- **Total: $18,250**

Database Tables

1. **locations** - Stores and expenses
2. **prices** - All pricing
3. **current_inventory** - Stock levels
4. **sales** - Ice cream sales
5. **inventory_purchases** - Bulk ice cream
6. **supply_purchases** - Containers/napkins

Conversions

- 1 gallon = 128 oz
- 1 container (5 gal) = 640 oz
- Low stock = 10 gallons = 1,280 oz